

Certificate Authority Chain and Deployment of a CA on a Linux Server

Network Security Project — Academic Report

Fernandez Ojeda Franklin 000541971
Lecky-Thompson William 000613648
Otto Aleksandra 000569128
Rafaaf Moheeb Eskandar Abanoub 000567614
Vanderslagmolen Moïra 000547486

June 7, 2026

Abstract

This report presents the design, deployment, and validation of a complete Public Key Infrastructure (PKI) on a Linux server using OpenSSL. Starting from the theoretical foundations of X.509 certificates and PKI trust models, it covers the full lifecycle of certificate management: creation of a two-tier Certificate Authority hierarchy (Root CA and Intermediate CA), issuance of server and client end-entity certificates, certificate revocation via CRL, and the deployment of Mutual TLS (mTLS) for bidirectional authentication. The practical implementation demonstrates how mTLS strengthens security beyond standard TLS by requiring both parties to present valid certificates during the handshake, and examines its relevance in modern Zero Trust architectures and microservice environments.

1. Introduction

1.1 Objectives

This report aims to achieve the following:

- Explain the structure and role of X.509 digital certificates in PKI.
- Describe the components and trust models of Public Key Infrastructure.
- Present the certificate signing and verification process in detail.
- Provide a practical guide to deploying a root CA and intermediate CA on a Linux server using OpenSSL.
- Demonstrate how to issue end-entity certificates for TLS services.
- Implement and validate Mutual TLS (mTLS) for bidirectional authentication between client and server.

1.2 Motivation

Secure communication on the Internet depends fundamentally on a trustworthy framework for managing digital identities. HTTPS, VPNs, email signing, and code signing all rely on X.509 certificates and Certificate Authorities. Yet the inner workings of PKI remain opaque to many practitioners. Beyond standard server authentication, the growing adoption of Zero Trust architectures makes client-side certificate authentication, as provided by mTLS, increasingly critical in modern deployments.

1.3 Report Structure

1. **Background:** X.509 certificates and the PKI framework.
2. **Certificate Signing & Verification:** full lifecycle

and verification checks.

3. **CA Deployment on Linux:** setting up root and intermediate CAs, issuing end-entity certificates, and configuring revocation.
4. **Mutual TLS:** theory, handshake mechanics, and use cases.
5. **Limitations of PKI:** structural and operational weaknesses illustrated by documented real-world incidents.

2. Background

2.1 X.509 Certificates

An X.509 certificate is a digital document that binds a public key to a verified identity (a person, server, organization, or device). The certificate is signed by a Certification Authority (CA), allowing relying parties to trust its contents without prior contact with the owner.

The X.509 standard defines three versions: v1, v2, and v3. Version 3 is universally adopted because it supports extensions that convey critical security metadata. Table 1 summarizes the core fields.

2.1.1 Version 3 Extensions

Version 3 extensions are one of the main reasons X.509 certificates are useful in real systems. The basic certificate fields identify the subject, issuer, and public key, but the extensions tell software *how* the certificate may be used and what limits apply to it. Each extension has an identifier, a value, and a *critical* flag. If an application does not understand a critical extension, it must reject the certificate rather than guess its meaning

Table 1: Core X.509 v3 Certificate Fields

Field	Description
Version	Format version (v1/v2/v3).
Serial Number	Unique identifier assigned by the CA.
Sig. Algorithm	Algorithm used to sign (e.g. SHA-256/RSA).
Issuer Name	Distinguished Name (DN) of the issuing CA.
Validity Period	The window of time in which the certificate is valid and trusted.
Subject Name	DN of the certificate owner.
Public Key Info	Owner's public key and algorithm.
Extensions	Key Usage, SAN, Basic Constraints, etc.
Signature	CA's cryptographic signature over TBSCert (the certificate body).

[1].

In practice, extensions let a CA describe whether a certificate is for a web server, a client, an intermediate CA, or a root CA. They also help software build chains correctly, check revocation information, and apply naming or policy restrictions. For this reason, extensions are not just optional extra data; they are central to certificate validation [1].

Some of the most important Version 3 extensions are the following:

- **Key Usage:** This extension lists the cryptographic operations allowed for the key, such as `digitalSignature`, `keyEncipherment`, `keyCertSign`, and `cRLSign`. For example, a CA certificate should allow certificate signing, while a normal TLS server certificate should not.
- **Extended Key Usage (EKU):** EKU gives a more precise purpose to the certificate, such as `serverAuth`, `clientAuth`, `codeSigning`, or `emailProtection`. In practice, both Key Usage and EKU should be consistent with the role of the certificate.
- **Subject Key Identifier (SKI) and Authority Key Identifier (AKI):** These two extensions help software link certificates together. SKI identifies the subject's key, while AKI points to the key of the issuing CA. They are especially useful when building and checking a certificate chain.
- **Certificate Policies:** This extension lists policy object identifiers (OIDs) that describe the rules under which the certificate was issued. Related extensions such as Policy Constraints and Inhibit Any Policy help control policy processing in more complex PKI environments.
- **Subject Alternative Name (SAN):** SAN can contain DNS names, IP addresses, email addresses,

URIs, and other identities. In modern TLS, SAN is the main place checked for hostname matching. RFC 5280 also notes that if the subject field is empty, the SAN extension must be present and marked critical [1].

- **Basic Constraints:** This extension states whether a certificate belongs to a CA by using `CA:TRUE` or `CA:FALSE`. It may also include `pathLenConstraint`, which limits how many subordinate CA levels may appear below that certificate.
- **Name Constraints:** This extension is mainly used in CA certificates to restrict the names that subordinate certificates may contain. For example, a CA can be limited to issuing certificates only for a specific domain or namespace.
- **CRL Distribution Points and Authority Information Access (AIA):** These extensions tell clients where to find revocation and issuer information. CRL Distribution Points give the location of the certificate revocation list, while AIA may point to issuer certificates or OCSP services used during validation.

Overall, Version 3 extensions make certificate validation more precise and more secure. They allow software to enforce the intended role of a certificate, distinguish between CA and end-entity certificates, and obtain supporting information such as revocation status and issuer details. Without these extensions, modern PKI would be much harder to manage safely [1].

Certificates are typically written in one of two formats:

- **Distinguished Encoding Rules (DER):** a binary encoding format.
- **Privacy Enhanced Mail (PEM):** Base64-encoded DER with header and footer lines, commonly used in configuration files.

2.2 Public Key Infrastructure (PKI)

PKI is the overarching framework for managing the issuance, distribution, validation, and revocation of digital certificates. Beyond cryptographic algorithms, PKI defines the policies, roles, and procedures that allow parties to establish trust across untrusted networks.

2.2.1 PKI Trust Models

- **Hierarchical (Single Root):** a single root CA delegates trust downward to subordinate CAs. Used by public web PKIs (e.g. browser root stores).
- **Cross-Certification (Mesh):** multiple independent CAs establish trust relationships by mutually issuing certificates to one another. Instead of relying on a single root authority, trust is distributed among participating organizations.
- **Bridge CA:** a Bridge Certification Authority acts as a central hub that connects multiple independent PKI domains. Rather than every CA cross-certifying with every other CA, each participating PKI only establishes trust with the bridge CA.

2.2.2 Main PKI Components

A Public Key Infrastructure (PKI) consists of several components that work together to establish, distribute, validate, and revoke digital trust.

- **Certificate Authority (CA):** the trusted entity responsible for issuing, digitally signing, renewing, and revoking certificates. The CA binds a subject's identity to its public key, thereby acting as the root of trust within the PKI hierarchy.
- **Registration Authority (RA):** an entity that performs identity verification on behalf of the CA. The RA validates information provided by certificate applicants before forwarding approved requests to the CA for certificate issuance.
- **Digital Certificates:** electronic documents conforming typically to the X.509 standard. A certificate contains identifying information about the subject, the subject's public key, validity period, certificate serial number, issuer information, and the CA's digital signature.
- **Certificate Signing Request (CSR):** a structured request generated by an applicant when requesting a certificate. The CSR contains the applicant's public key and identifying information (such as domain name or organization details) and is digitally signed using the applicant's private key to prove possession of that key.
- **Certificate Repository:** a centralized directory service (commonly LDAP-based) used to store and distribute issued certificates, CA certificates, and Certificate Revocation Lists (CRLs). It allows relying parties to retrieve certificates required for validation.
- **Revocation Services:** mechanisms used to determine whether a certificate is still trusted before its expiration date. Common methods include Certificate Revocation Lists (CRLs) and the Online Certificate Status Protocol (OCSP).
- **Certificate Policy (CP) / Certification Practice Statement (CPS):** formal documents defining the PKI's security requirements, trust model, operational procedures, identity verification processes, and certificate management practices.

2.2.3 PKI Operation Sequence

The certificate issuance and validation lifecycle within a PKI typically follows the sequence below:

1. Generate an asymmetric cryptographic key pair consisting of a public key and a private key.
2. Create a Certificate Signing Request (CSR) containing the public key and subject identity information. The CSR is digitally signed using the private key to prove ownership and possession of the corresponding key.
3. Submit the CSR to a Registration Authority (RA) or directly to the CA.
4. The RA verifies the applicant's identity according to the PKI policy and forwards the validated request to the CA.
5. The CA validates the request and digitally signs the certificate using its private key, thereby binding the applicant's identity to the public key.
6. The signed certificate is issued and delivered to the requester, who installs it on a server, device, application, or user system.
7. The certificate may also be published to a certificate repository so that relying parties can retrieve it when necessary.
8. During communication, relying parties validate the certificate by checking the CA signature, certificate chain, validity period, hostname or subject information, and revocation status using CRL or OCSP services.
9. If a certificate is compromised, expired, or no longer trusted, the CA revokes it and updates revocation information accordingly.

3. Certificate Signing and Verification

The X.509 PKI framework, as standardized in ITU-T Recommendation X.509 and RFC 5280 [1], establishes a hierarchical trust model wherein digital certificate signing serves as the fundamental mechanism for authenticating cryptographic identities. Through the application of asymmetric cryptography, Certification Authorities generate unforgeable digital signatures that cryptographically bind entity identities to their respective public keys. This process, which involves the generation of a signed ASN.1-encoded TBSCertificate structure, provides both data origin authentication and message integrity through mathematical verification of the CA's signature using standardized algorithms such as RSA-PKCS#1 v1.5 or ECDSA [1].

The verification process constitutes a critical component of secure communication protocols, including the TLS handshake [4]. During certificate chain validation, relying parties must not only verify the digital signature's mathematical correctness but also enforce temporal validity constraints, key usage policies, and path validation rules as specified in RFC 5280 Section 6. This multi-faceted verification process ensures that only properly authorized entities can participate in secure communications, while effectively mitigating well-known attacks such as man-in-the-middle (MITM) interposition.

3.1 Certificate Signing Process

3.1.1 Key Pair Generation

Before creating a CSR, the end-entity must generate an asymmetric key pair using cryptographic software. The private key is kept secret to sign documents and decrypt data, while the public key is included in the CSR. Two principal algorithms are used:

- **RSA:** used for both encryption and digital signatures. Recommended key sizes are 2048 bits (minimum) or 4096 bits for longer-lived certificates.
- **ECDSA:** a signature-only algorithm offering equiv-

alent security with shorter keys. Curve P-256 is standard for TLS certificates [1].

Some good practices can be adopted to ensure the secure storage of the private key:

- Restrictive file permissions on Linux.
- Key encryption with a strong passphrase.

3.1.2 CSR Structure (PKCS#10)

A Certificate Signing Request is the formal process of applying for a digital certificate from a Certificate Authority. The CSR structure is defined in PKCS#10 [5], a security standard produced by RSA Data Security Inc. It consists of a subject (DN info), the subject public key and its algorithm, and a set of attributes. These fields are encoded using ASN.1 (Abstract Syntax Notation 1) and DER (Distinguished Encoding Rules), which are binary encoding standards used in X.509.

A CSR contains three main parts:

- **Certification Request Information:** contains pertinent information such as the public key, Common Name, Organization Name, Organizational Unit, Country, key type, and key size.
- **Signature Algorithm Identifier:** specifies the algorithm used to sign the CSR.
- **Digital Signature:** proves possession of the private key. The requester signs the CSR with their private key, allowing the CA to verify they control it.

The information provided in the CSR must be accurate and match the existing domain registration. Otherwise, certificate issuance delays or even rejections may occur.

3.1.3 Submission and Identity Validation

Once the CSR is completed, it is submitted to a trusted Certificate Authority, which can be either internal (e.g. a corporate PKI) or external (e.g. Let's Encrypt or DigiCert). The CA performs identity verification, and the exact process depends on the type of certificate requested:

- **Domain Validation (DV):** the CA verifies only that the applicant controls the domain name. Verification is usually automated through an email challenge, a DNS TXT record, or an HTTP file upload. DV certificates are fast and inexpensive to issue, but they do not verify the identity of the organization behind the domain.
- **Organization Validation (OV):** the CA verifies both domain ownership and the legal existence of the organization. This may include checking business registries, verifying the organization's name, address, and registration in official databases, and conducting phone verification with authorized personnel. OV certificates provide stronger identity assurance than DV certificates.
- **Extended Validation (EV):** the CA performs strict identity verification, including legal, physical, and operational checks of the organization. There

must be confirmation that the person requesting the certificate is authorized to act on behalf of the organization. EV certificates provide the highest traditional level of trust and are commonly used by banks, governments, and large organizations.

3.1.4 Certificate Issuance

Once validation passes, the CA constructs the TBSCertificate structure in ASN.1, populates all fields and extensions, computes a cryptographic hash, and signs it with the CA private key using the chosen algorithm (e.g. SHA-256/RSA or ECDSA P-256). The complete CSR structure is digitally signed by the CA using the CA's private key and a chosen signature algorithm. The signed certificate is then returned to the requester in PEM or DER format.

3.2 Certificate Verification Process

To ensure that a digital certificate is valid, the client must proceed through a multi-step verification process [1]. This involves signature validation, chain verification, and revocation status checks. Skipping any step opens the door to attacks such as MITM or certificate substitution.

3.2.1 Basic Certificate Checks

- **Validity Period:** a certificate contains a Not Before and a Not After timestamp. The verifier must check that the certificate is within its active date range.
- **Subject and Issuer Name:** the Subject field must match the entity, and the Issuer field must match the expected Certificate Authority.
- **Key Usage and Extended Key Usage (EKU):** the verifier must ensure that the certificate is used for its intended purpose. For example, a TLS server certificate should have Server Authentication in its EKU.

3.2.2 Digital Signature Verification

Every certificate is digitally signed by its issuing CA. The signature proves that the CA has validated the certificate's contents. First, the certificate's signed data and signature are extracted. The issuer's public key is then used to verify the signature mathematically. If the signature is valid, it confirms that the certificate was issued by that CA and that its content was not modified since it was signed.

3.2.3 Certificate Chain Validation

Most certificates are not directly signed by a root CA but by intermediate CAs. The verifier must validate the entire chain of trust [1].

The process starts with the end-entity certificate belonging to the website or server. The verifier retrieves the issuer's certificate, or intermediate CA certificate, and repeats this process until a trusted root CA is reached. For each certificate in the chain, the verifier must:

1. Verify the digital signature.

2. Check the validity period.
3. Ensure no revocation (via CRL or OCSP).
4. Confirm basic constraints (e.g. intermediate CA must have `CA:TRUE`).
5. Confirm the root CA exists in the client's trust store.

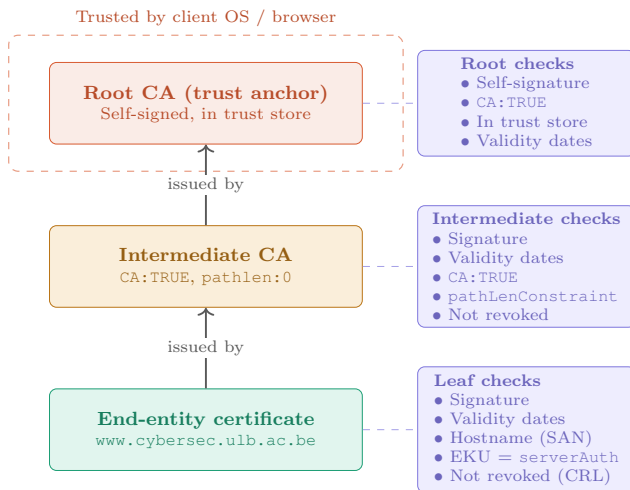


Figure 1: Certificate chain validation. The verifier starts at the end-entity certificate and walks upward to a trusted root, performing the listed checks at each level. The chain is only as strong as its weakest link: a single failure at any tier invalidates the whole connection.

Security note: A chain is only as strong as its weakest link. If an intermediate CA is compromised, all certificates below it must be revoked. This is why root CAs are kept *offline* and intermediate CAs handle day-to-day operations.

3.2.4 Revocation Status Checking

Even if a certificate is properly signed, it may have been revoked due to compromise or expiration. Two main revocation methods exist:

- **CRL (Certificate Revocation List):** a list of revoked certificates published by the CA. The verifier downloads this CRL and checks if the certificate's serial number is listed.
- **OCSP (Online Certificate Status Protocol):** a real-time query to the CA to check certificate status [3]. It returns Good, Revoked, or Unknown. If a certificate is found in the CRL or marked as revoked, it is not trusted. *OCSP Stapling* allows the server to cache and present a signed OCSP response, eliminating client round-trips and improving privacy.

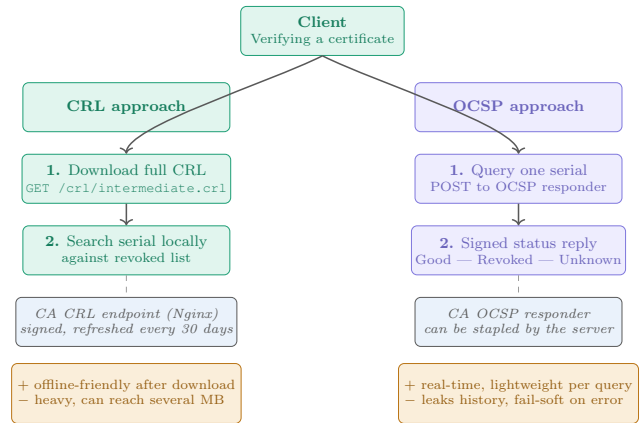


Figure 2: Two revocation-checking strategies compared. The CRL approach (left, used in our deployment) downloads the CA's signed revocation list and matches the certificate's serial number locally. The OCSP approach (right) queries the CA in real time for a single serial. Both are required by the validation steps of Figure 1, and our renewal pipeline regenerates the CRL weekly to avoid the failure mode described in Section 5.

3.2.5 Hostname Verification

For HTTPS and other TLS connections, the certificate's Common Name (CN) or Subject Alternative Name (SAN) must match the domain being accessed [2]. For example, if a user visits `https://example.com` and the certificate is issued for `example.org`, the connection will be rejected. Modern clients rely primarily on the SAN extension for this check, as the legacy CN field is no longer used for hostname matching in current browsers.

3.3 Role in TLS Secure Communication

X.509 certificates are used to secure communications between a client (such as a web browser) and a web server. When a user connects to a website using HTTPS, the web server presents a digital certificate compliant with the X.509 standard. This certificate includes information about the server's identity (domain name), its public key, and a digital signature issued by a Certificate Authority. The client then verifies the validity of the certificate by walking up the certificate chain to a pre-installed trusted root authority in the operating system or browser.

During a TLS 1.3 handshake [4], the server transmits its certificate chain immediately after the Server Hello. The client performs all the checks described above, and only on full success does it proceed to derive session keys. This mechanism provides:

- **Authentication:** the user can be sure they are communicating with the legitimate website and not an impostor.
- **Confidentiality:** data exchanged cannot be read or intercepted by third parties.
- **Integrity:** any alteration in the transmitted messages can be detected.

As such, X.509 certificates are essential for securing web traffic, preventing attacks such as man-in-the-middle and DNS spoofing, and establishing trust between users and online services.

4. CA Deployment on a Linux Server

4.1 Architecture Overview

A standard deployment uses a **two-tier hierarchy**:

- **Root CA** — kept offline on an air-gapped machine or HSM. Only used to sign the intermediate CA certificate.
- **Intermediate CA** — online, used for day-to-day certificate issuance. Compromise of the intermediate CA does not expose the root key.

The remainder of this section follows a two-tier deployment; the complete, runnable procedure—every OpenSSL invocation, the Nginx configuration, and the renewal scripts—is provided in the companion [Implementation Guide](#), to which each subsection below links.

Best practice: generate and store the root CA private key on an air-gapped machine or HSM. Bring it online *only* to sign intermediate CA certificates. Store backups in separate physical locations.

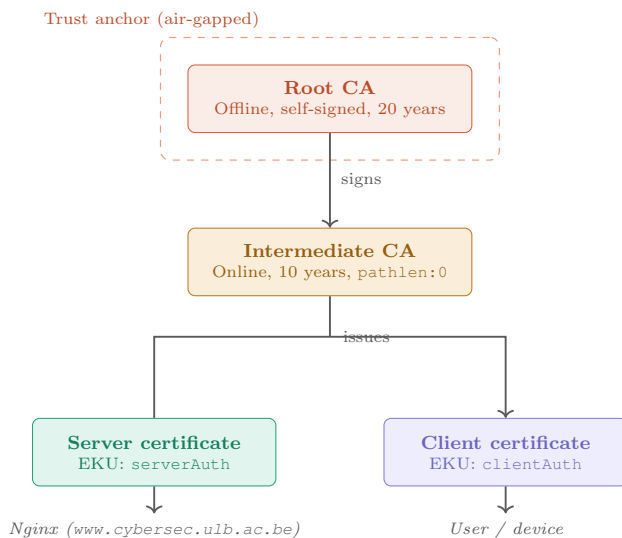


Figure 3: Two-tier Certificate Authority architecture. The Root CA is kept offline and only signs the Intermediate CA, which in turn issues server and client end-entity certificates used in our mTLS deployment.

4.2 Directory Structure

A well-organized directory structure is essential for auditability. The working directory for the root CA should contain the following (the commands that create this layout are given under [Directory Structure](#) in the [Implementation Guide](#)):

- `certs/` — issued certificates.

- `crl/` — Certificate Revocation Lists.
- `newcerts/` — archive of all signed certificates.
- `private/` — private keys, with restricted permissions.
- `index.txt` — the certificate database.
- `serial` — the next serial number to be assigned.
- `openssl.cnf` — the root CA configuration file.
- `intermediate/` — the intermediate CA, with the same layout.

4.3 Step-by-Step Deployment

4.3.1 Step 1 — Create the Root CA

The first step is to generate the root CA private key, protected with AES-256 encryption. A self-signed root certificate is then created with a long validity period (typically 20 years), using the `v3_ca` extensions defined in the configuration file. The resulting certificate should be verified to confirm that all fields and extensions are correct. The exact OpenSSL commands for these operations are listed in [Steps 1–3 of the Implementation Guide](#).

4.3.2 Step 2 — Create the Intermediate CA

An intermediate CA key is generated in the same way as the root key. A CSR is then created for the intermediate CA and submitted to the root CA for signing. The root CA signs the intermediate certificate using the `v3_intermediate_ca` extensions, which include `CA:TRUE` and an appropriate `pathLenConstraint`. Finally, a certificate chain bundle is assembled by concatenating the intermediate certificate and the root certificate into a single file, which is provided to relying parties during TLS handshakes. These operations correspond to [Steps 4–7 of the Implementation Guide](#).

4.3.3 Step 3 — Issue End-Entity Certificates

A server key and CSR are generated for the target service. The CSR must include the Subject Alternative Name (SAN) extension specifying the domain names or IP addresses the certificate will cover. The intermediate CA signs the certificate using the `server_cert` extensions, which set `CA:FALSE`, restrict key usage to `digitalSignature` and `keyEncipherment`, and set the Extended Key Usage to `serverAuth`. The signed certificate, private key, and chain bundle are then deployed to the web server (e.g. Nginx or Apache). The server- and client-certificate commands are given in [Steps 8–9 of the Implementation Guide](#).

4.3.4 Step 4 — Configure Revocation (CRL & OCSP)

An initial CRL is generated and published at the URL specified in the `crlDistributionPoints` extension of issued certificates. When a certificate must be revoked, it is marked as revoked in the CA database and the CRL is regenerated and republished. For real-time revocation queries, an OCSP responder is deployed and its URL is included in the Authority Information Access (AIA) extension of issued certificates. The CRL generation, revocation, and distribution commands are

provided in [Steps 10–12 of the Implementation Guide](#).

4.4 Key OpenSSL Configuration

The `openssl.cnf` file controls all CA behavior. The `[ v3_ca ]` section defines the extensions applied to CA certificates, including `CA:TRUE`, key usage for certificate and CRL signing, and key identifier extensions. The `[ server_cert ]` section defines end-entity certificate extensions, setting `CA:FALSE`, restricting key usage, and requiring a `subjectAltName` entry.

The complete `openssl.cnf` files for the root and intermediate CAs, including these sections, are reproduced in [Step 1](#) and [Step 4](#) of the [Implementation Guide](#).

SAN is mandatory. Modern browsers reject certificates that lack a Subject Alternative Name, even if the Common Name matches. Always include `subjectAltName` in server certificate configurations.

participate without a valid certificate issued by the trusted CA.

This bidirectional trust is particularly relevant in environments where the identity of the connecting entity, not just the confidentiality of the channel, must be cryptographically guaranteed.

5.2 mTLS Handshake Mechanics

Certificate

IT departement cert	ULB cybersecurity Intermediate CA	ULB cybersecurity Root CA
Subject Name		
CountryBE		
State/ProvinceBrussels		
LocalityBrussels		
OrganizationULB Cybersecurity Lab		
Organizational UnitIT Department		
Common NameIT departement cert		
Email Addressadmin@cybersec.ulb.ac.be		
Issuer Name		
CountryBE		
State/ProvinceBrussels		
OrganizationULB Cybersecurity Lab		
Organizational UnitIntermediate CA		
Common NameULB cybersecurity Intermediate CA		
Email Addressintermediateca@cybersec.ulb.ac.be		
Validity		
Not BeforeSat, 09 May 2026 07:26:22 GMT		
Not AfterWed, 19 May 2027 07:26:22 GMT		

Figure 4: Certificate chain

5. Mutual TLS (mTLS)

5.1 Definition and Motivation

Standard TLS provides *one-way* authentication: the server presents a certificate to the client, which verifies it against a trusted root CA. The client, however, remains anonymous to the server. This is sufficient for public-facing websites, where the server does not need to know *who* is connecting—only that the communication is confidential.

Mutual TLS (mTLS) extends this model to *two-way* authentication. Both the server and the client present X.509 certificates, and both verify each other’s certificate against a trusted CA before the connection is established. Neither party can impersonate the other, and an attacker who intercepts the channel cannot par-

In a standard TLS 1.3 handshake [4], the server sends its certificate chain after the `ServerHello` message. The client verifies it and, on success, derives session keys. In mTLS, the server additionally sends a `CertificateRequest` message, which signals that the client must also authenticate. The extended handshake proceeds as follows:

1. The client sends `ClientHello`.
2. The server responds with `ServerHello`, its certificate chain, and a `CertificateRequest`.
3. The client verifies the server certificate against its trust store.
4. The client sends its own certificate chain in response to the `CertificateRequest`.
5. The client sends a `CertificateVerify` message, containing a signature over the handshake transcript produced with its private key. This proves that the client possesses the private key corresponding to the public key in its certificate.
6. The server verifies the client certificate against its configured CA and checks the `CertificateVerify` signature.
7. Both parties derive session keys and the connection proceeds.

If either certificate fails validation, due to an invalid signature, expired validity, revoked status, or missing `clientAuth` in the EKU of the client certificate, the handshake is aborted and the connection is rejected.

The server-side configuration that enforces this exchange (`ssl_verify_client on`) and the `curl` tests that validate it are given in [Steps 13–14 of the Implementation Guide](#).

Key extension: client certificates used in mTLS must include `clientAuth` in their Extended Key Usage field. In our deployment, this is set in the `[ client_cert ]` block of `openssl.cnf` ([Step 4 of the Implementation Guide](#)).

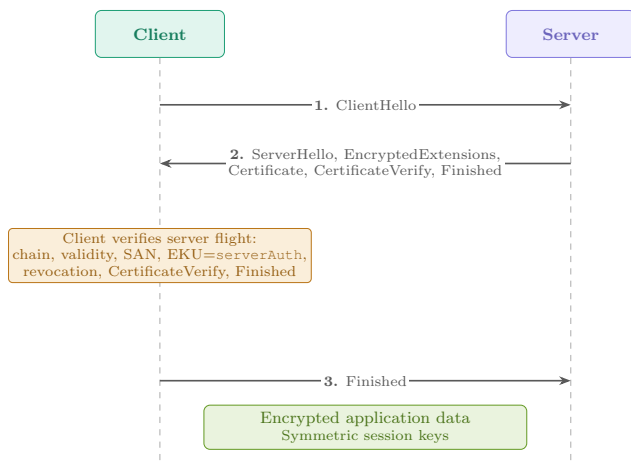


Figure 5: Standard TLS 1.3 handshake. Only the server presents a certificate; the client remains anonymous. The dashed lines represent the client and server lifelines.

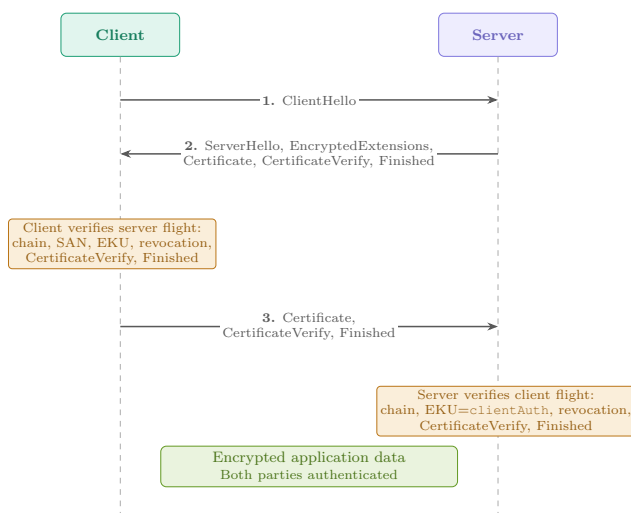


Figure 6: Mutual TLS handshake. In addition to the standard TLS exchange, the server issues a CertificateRequest, the client presents its own certificate and a CertificateVerify signature, and the server validates the client certificate against the configured CA chain before deriving session keys.

5.3 Comparison: TLS vs. mTLS

Table 2: TLS vs. mTLS comparison

	TLS	mTLS
Server auth	Yes	Yes
Client auth	No	Yes (certificate)
Client anon.	Yes	No
Handshake	Standard	Extended (+CertReq)
Typical use	Public websites	APIs, microservices, VPNs
Revocation	Server cert	Both certs

5.4 Relevance in Modern Network Security

mTLS has become a foundational building block in several current security architectures:

- **Zero Trust Architecture:** the Zero Trust model assumes that no entity—inside or outside the network perimeter—is trusted by default. Every connection must be authenticated and authorized explicitly. mTLS provides the cryptographic identity layer that enables this, ensuring that even an attacker who has gained access to the internal network cannot establish connections without a valid certificate.
- **Microservice communication:** in distributed systems, services communicate over internal networks that may be shared or untrusted. Service meshes such as Istio and Linkerd use mTLS transparently between every pair of services, so that each service can verify it is talking to a legitimate peer and not a compromised component.
- **API security:** enterprise APIs increasingly require client certificates instead of—or in addition to—API tokens. Certificate-based authentication is stronger than bearer tokens because the private key never leaves the client and cannot be stolen from a database.
- **IoT device authentication:** in IoT deployments, each device is provisioned with a unique certificate at manufacture time. When the device connects to a backend server, mTLS ensures that only legitimate, enrolled devices are accepted, preventing rogue device injection.

5.5 Certificate Validity and Renewal

One operational challenge of mTLS is certificate lifecycle management. Certificate authorities and browser requirements have progressively shortened maximum validity periods: from several years before 2015, to 398 days by 2020, and proposals from major vendors now target 47 days or fewer [1]. Short-lived certificates limit the window of exposure if a private key is compromised, but they require automated renewal processes to avoid service disruption.

In our deployment, renewal is handled by a Bash script executed daily via cron, the Linux task scheduler (Step 17 of the [Implementation Guide](#), scheduled in Step 19). The script is idempotent: it inspects every certificate in its inventory and triggers a renewal only when the remaining validity falls below a 30-day threshold, mirroring the behaviour of Let’s Encrypt’s certbot. When renewal is required, the script generates a fresh key pair and a new CSR, submits it to the Intermediate CA, and verifies the resulting certificate against the chain *before* swapping any live files. The previous certificate and key are first copied to a time-stamped backup directory, and Nginx is reloaded only after `nginx -t` confirms that the new configuration is valid. If any of these checks fails, the previous certificate remains in place and the service is not disrupted.

A separate weekly job regenerates the Certificate Revocation List well before its 30-day `nextUpdate` boundary (Step 18 of the Implementation Guide), since an expired CRL would cause Nginx to reject every connection regardless of the validity of the end-entity certificates. The two jobs are kept on independent schedules because they concern different state—the end-entity certificate inventory on one side, the revocation database on the other—and decoupling them keeps each script focused on a single responsibility.

For unattended execution, the Intermediate CA passphrase is stored in a root-owned file with 0600 permissions and supplied to OpenSSL via the `-passin file: option` (Step 16 of the Implementation Guide). This is a deliberate trade-off: a passphrase file on the same machine reduces protection against host compromise, but it allows the entire renewal pipeline to run without human intervention. In a production environment, the Intermediate CA key would be stored in a Hardware Security Module or a Key Management Service so that the key material never leaves the secure boundary.

Operational note: in mTLS deployments, *both* the server and client certificates must be renewed before expiry. An expired client certificate causes the same handshake failure as an invalid one. The script therefore iterates over the complete certificate inventory rather than only the server side, and each entry declares its own extension block (`server_cert` or `client_cert`) so that the correct Extended Key Usage is preserved across renewals.

5.6 Limitations of mTLS

Despite its security benefits, mTLS introduces operational complexity that must be carefully managed:

- **Certificate distribution:** every client must be provisioned with a valid certificate before it can connect. In large deployments this requires a scalable enrollment process, often handled by an automated PKI management system.
- **Revocation propagation:** if a client certificate is compromised, it must be revoked and the CRL or OCSP responder updated promptly. Until the revocation is published, the compromised certificate remains usable.
- **Compatibility:** standard web browsers do not present client certificates automatically. mTLS is therefore suited to machine-to-machine communication rather than general user-facing web traffic.
- **Overhead:** the additional certificate exchange and verification steps add a small latency cost to connection establishment, though this is negligible for most applications.

6. Limitations of the PKI Model

Despite the strong cryptographic foundations of X.509 and the maturity of modern PKI tooling, the global Certificate Authority ecosystem has suffered recurring

failures over the past fifteen years. The limitations discussed below are not theoretical: each is illustrated by a well-documented real-world incident, and each motivated specific design choices in our own deployment. They show that the weak points of PKI lie not in its mathematics but at its edges—operations, human processes, governance, and revocation infrastructure.

6.1 Technical Compromise of a CA

A legitimate CA can be breached through poor operational security: outdated servers, insufficient network segmentation, or weak monitoring. If a single CA present in the browser trust store is compromised, an attacker can forge *valid certificates for any domain* (Google, banks, governments), and victim browsers will accept them silently. This enables large-scale man-in-the-middle attacks on HTTPS.

Case study — DigiNotar (2011). The Dutch CA DigiNotar was breached through an obsolete public-facing web server and insufficient internal network segmentation. The attacker issued **531 fraudulent certificates**, including a wildcard `*.google.com`, which was used to intercept the Gmail traffic of approximately **300,000 Iranian citizens** in what was likely a state-sponsored MITM operation. All major browsers distrusted DigiNotar within weeks, and the company **filed for bankruptcy in September 2011**¹.

This incident demonstrates that cryptographic strength offers no protection against poor operational hygiene. It is precisely the reason our deployment keeps the Root CA *offline* and uses an online Intermediate CA only for day-to-day operations.

6.2 Commercial Pressure and “Too Big to Fail” CAs

Public CAs are commercial enterprises that sell certificates at scale, and this economic pressure can lead to lax validation, poorly supervised resellers, and the dismissal of internal errors. When a CA becomes dominant on the market, sanctioning it becomes politically and technically expensive: a significant fraction of the web could be rendered inaccessible overnight.

Case study — Symantec distrust (2017–2018). Symantec, together with its VeriSign, Thawte, and GeoTrust brands, issued **more than 30% of all valid web certificates** at the time. A Google Chrome investigation initially identified 127 misissued certificates, but the total grew to at least **30,000** over several years, many of them emitted through unsupervised resellers. Google announced a progressive distrust of Symantec certificates in Chrome 66 and Chrome 70, and Symantec ultimately **sold its entire CA division to DigiCert for approximately \$1 billion**².

¹<https://en.wikipedia.org/wiki/DigiNotar>; see also Fox-IT, *Operation Black Tulip: Report of the investigation into the DigiNotar Certificate Authority breach*, 2012.

²<https://www.thesslstore.com/blog/google-chrome-distrust-symantec-ssl-certificates/>; original announcement on the Chromium blog: *Intent to Depre-*

This case illustrates that the commercial PKI model has a structural conflict of interest, and that market concentration weakens the ecosystem’s ability to sanction misbehaviour.

6.3 Revocation Is Technically Broken

The two main mechanisms for checking whether a certificate has been revoked, CRL and OCSP, have flaws that make them effectively unusable on the public web:

- **CRL** files list every revoked certificate and can grow to several megabytes, making them impractical to download on every connection.
- **OCSP** performs a real-time query to the CA, which is slow and leaks the user’s browsing history to the CA [3].
- If the OCSP request fails—due to an outage or to an attacker actively blocking it—browsers **fail soft**: they accept the certificate anyway to avoid breaking the web. A network-positioned attacker can therefore neutralise revocation simply by dropping the OCSP request.

Case study — Heartbleed (April 2014). The Heartbleed vulnerability in OpenSSL exposed TLS private keys on approximately **17% of HTTPS servers worldwide**, forcing thousands of operators to revoke their certificates. However, Chrome had already disabled OCSP by default, and its alternative mechanism (CRLSets) contained only about **24,000 entries out of more than 2 million revoked certificates**—roughly **98% ignored**³. An attacker who had stolen a private key could keep using it even after formal revocation.

This connects directly to Section 4 (CRL configuration) and motivates the broader industry trend of shortening certificate validity periods (398 days, then 90 days, with 47 days now proposed), as well as the automatic renewal script described in Section 5 and implemented in Step 17 of the Implementation Guide.

6.4 Default Trust and CA Governance

Modern browser trust stores contain 150–200 CAs that users trust implicitly, without ever being asked to choose. Several of these CAs operate under jurisdictions that can compel them to cooperate. Unlike the previous case of an accidentally compromised CA, here the CA itself is the threat: it may *deliberately* issue fraudulent certificates, or be forced to, and the bare X.509 model provides no native mechanism to restrict which CA may issue for which domain.

Case study — CNNIC / MCS Holdings (2015). CNNIC, China’s official CA, was present in every major trust store. CNNIC issued an **intermediate certificate to MCS Holdings**, an Egyptian company

with no legitimate CA infrastructure, and **without any name constraint**, meaning MCS could issue certificates for arbitrary domains. MCS produced **fake Google certificates** and installed them in a corporate firewall to intercept its employees’ encrypted traffic. The incident was detected through **Certificate Transparency**, after which Google and Mozilla distrusted CNNIC⁴.

This case shows that “default” trust is a structural weakness of the model. It directly motivated **Certificate Transparency** (RFC 6962), which forces CAs to publish every issued certificate to public, append-only logs, making large-scale misissuance detectable.

6.5 Human Factor: Employees and Resellers

CA security ultimately depends on humans: administrators, employees, resellers. All cryptographic safeguards are bypassed if an administrative account is phished or if a partner reuses credentials. In practice, attacks on CAs almost always exploit the human factor rather than break the underlying cryptography.

Case study — ComodoHacker (2011). A solo attacker calling themselves “ComodoHacker” compromised a **Comodo reseller account based in Italy** by stealing valid credentials. There was no sophisticated exploit and no cryptographic break—only a poorly protected account at a business partner. The attacker issued **nine fraudulent certificates** for Google, Yahoo, Skype, Mozilla, and login.live.com, and claimed the operation was politically motivated, drawing comparisons with the later DigiNotar attack⁵.

The modern response to this class of attack relies on mandatory multi-factor authentication, strict reseller audits, and real-time monitoring of certificate issuance, complemented by Certificate Transparency logs as a public check.

6.6 Operational Errors on the Operator Side

Even in the absence of any attack, organisations routinely compromise themselves: forgotten renewals, leaked private keys, and TLS misconfigurations are common. The larger the deployment—hundreds or thousands of certificates—the higher the probability of error.

Case study — Equifax (2017). The US credit-rating agency Equifax used an HTTPS-aware intrusion detection system that inspected encrypted traffic for attacks. The internal certificate enabling this inspection **expired and was not renewed for ten months**. During that period, the detection tool was effectively blind, and attackers exfiltrated the personal data of **145 million individuals** (Social Security numbers,

cate and Remove: Trust in existing Symantec-issued Certificates.

³<https://www.netcraft.com/blog/certificate-revocation-why-browsers-remain-affected-by-the-fakebleed4/certificates>

see also A. Langley, *Revocation doesn’t work*, ImperialViolet blog, March 2011: <https://www.imperialviolet.org/2011/03/18/revocation.html>

⁴<https://security.googleblog.com/2015/03/maintaining-digital-certificate-security.html>; see also https://en.wikipedia.org/wiki/CNNIC#Issuance_of_fake_certificates

⁵https://en.wikipedia.org/wiki/Comodo_Cybersecurity#Certificate_hacking; see also Mozilla Security Blog, *Comodo Certificate Issue Follow-up*, March 2011.

dates of birth, addresses) without triggering any alert. No CA was breached, no cryptography was broken—a single forgotten certificate was enough⁶.

This is precisely the failure mode that the **automatic renewal script** described in Section 5 ([Step 17 of the Implementation Guide](#)) is designed to prevent, and it is also the philosophy behind tools such as `certbot` for Let's Encrypt.

Synthesis: the recurring pattern across these incidents is that PKI breaks at its edges—operational hygiene, human processes, governance, and revocation infrastructure—rather than in its mathematical core. The architectural choices in our deployment (offline Root CA, automated renewal, scheduled CRL regeneration, restrictive file permissions, strict ECU separation between server and client certificates) are direct responses to these documented failure modes.

7. AI Strategy

7.1 Tools Used

Throughout this project, the primary AI tool used was **Claude** (Anthropic). It was used in three distinct roles: assistance with implementation, support for report writing, and guidance for information retrieval with verifiable sources.

7.2 Implementation Assistance

For the practical deployment of the CA infrastructure, mTLS configuration, and the certificate renewal script, Claude was used as a technical assistant rather than a code generator. The approach followed was to provide the AI with the full context of the task—the existing directory structure, the relevant `openssl.cnf` sections, and the precise objective—and to ask for targeted help on specific steps, such as the correct flags for a given OpenSSL command or the structure of a particular Nginx directive.

Every command and script produced with AI assistance was subsequently executed and tested on the actual deployment environment. Understanding the output was a requirement: if a command worked but its behaviour was not fully understood, it was not accepted. This ensured that the implementation reflects genuine understanding rather than blind execution of generated code. The resulting, fully tested procedure is documented end-to-end in the companion [Implementation Guide](#).

7.3 Report Writing

Claude was also used to support the writing of this report. Its role was limited to reformulation and grammar correction: sections were drafted by the group, then submitted to the AI with instructions to improve clarity, fix grammatical errors, or rephrase awkward sentences—while preserving the original meaning and

technical content. The ideas, structure, and technical judgements in this report are entirely our own. Re-reading each revised paragraph against the original draft ensured that no content was altered beyond stylistic improvement and that no hallucinated or inaccurate statements were introduced.

7.4 Information Retrieval and Source Verification

When researching topics such as the mTLS handshake mechanics, the evolution of certificate validity periods, or the role of mTLS in Zero Trust architectures, Claude was asked to provide not only explanations but also references to primary sources: RFCs, official documentation, and peer-reviewed material. Information obtained this way was cross-checked against the cited sources before being included in the report. For example, the description of the TLS 1.3 handshake in Section 5 was verified against RFC 8446 [4], and the X.509 extension semantics were checked against RFC 5280 [1]. Any claim that could not be traced to a verifiable source was either removed or reformulated with appropriate hedging.

7.5 Prompt Optimisation

A key lesson learned early in the project was that the quality of AI output is directly proportional to the precision and completeness of the prompt. Vague or open-ended prompts tend to produce generic answers that require significant correction. The approach adopted was therefore to make every prompt as specific as possible, providing all relevant context upfront: the current state of the deployment, the exact file or configuration section in question, the error message or behaviour observed, and the precise expected outcome. Rather than asking the AI to *find* a solution, prompts were structured so that the AI had all the information needed to *confirm or refine* a solution already partially formed. This reduced the risk of hallucination and kept the AI's role firmly in the position of assistant rather than decision-maker.

Example — imprecise prompt: “How do I configure mTLS?”

Example — precise prompt: “We have an Nginx server with a two-tier PKI (Root CA and Intermediate CA built with OpenSSL). The client certificate was issued using the `client_cert` extension block with `clientAuth` in ECU. What is the minimal Nginx configuration to enforce mandatory client certificate verification using `ssl_verify_client on`, pointing to our `ca-chain.cert.pem` file, with a verify depth of 2?”

7.6 Contribution to Exam Preparation

Beyond the practical tasks, Claude served as a personal tutor during the study phase. Concepts encountered in the course slides that were not immediately clear—such as the role of `pathLenConstraint`, the difference between CRL and OCSP Stapling, or the mechanics of the mTLS handshake—were explored interactively. Unlike a static textbook, the AI could adjust its level

⁶US House Committee on Oversight and Government Reform, *The Equifax Data Breach*, December 2018, p. 3: <https://republicans-oversight.house.gov/wp-content/uploads/2018/12/Equifax-Report.pdf>

of explanation, provide concrete analogies, and answer follow-up questions immediately. This made it a useful complement to the lecture material when preparing for the individual questions expected during the oral defence.

7.7 Limitations and Critical Assessment

The use of AI assistance also highlighted several limitations that required active management:

- **Hallucination risk:** the AI occasionally produced plausible but incorrect technical details, particularly for specific OpenSSL command flags or Nginx directive names. Systematic testing of every command in the actual environment was the primary mitigation.
- **Outdated information:** AI models have a knowledge cutoff and may not reflect the most recent changes in standards or tooling. For time-sensitive facts—such as current maximum certificate validity periods—primary sources and recent official documentation were always consulted.

References

-
- [1] D. Cooper et al., *Internet X.509 PKI Certificate and CRL Profile*, RFC 5280, IETF, May 2008.
 - [2] E. Rescorla, *HTTP Over TLS*, RFC 2818, IETF, May 2000.
 - [3] S. Santesson et al., *Online Certificate Status Protocol — OCSP*, RFC 6960, IETF, June 2013.
 - [4] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, IETF, August 2018.
 - [5] RSA Laboratories, *PKCS #10: Certification Request Syntax Standard*, Version 1.7, 2000.
 - [6] ITU-T, *Recommendation X.509: The Directory — Public-Key and Attribute Certificate Frameworks*, 2019.
-